

RFC-0006

Arquitetura de Autorização Granular e Controle de Acesso Dinâmico

Status: Aprovado e Atualizado (v1.2)

Autor: Equipe de Arquitetura

Data: 07/05/2026

Depende de: RFC-0001, RFC-0002, RFC-0005, Arquitetura de Identidade e Acesso Segura

1. Objetivo

Definir a arquitetura de autorização **extremamente granular**, permitindo:

- Controle por **endpoint + método HTTP**.
- Controle por **atributos do recurso (field-level security)**.
- Controle por **usuário individual (ABAC / PBAC)**.
- Governança centralizada e auditável.
- Total autonomia de domínio (isolamento estrito do Keycloak e proteção de dados LGPD).

Este RFC complementa o modelo híbrido RBAC+ABAC (RFC-0002), introduzindo o Motor Interno de Políticas (Policy Decision Point - PDP) gerido inteiramente pela aplicação e apoiado pelo banco de dados PostgreSQL.

2. Problema

O modelo tradicional baseado apenas em roles genéricas do Provedor de Identidade (Keycloak):

- Causa inchaço do token JWT com dados sensíveis de permissões organizacionais.
- Não permite variações finas por usuário dentro de contextos multi-tenant.
- Não controla campos específicos na resposta da API.
- Não diferencia permissões por contexto físico e temporal (horário, geolocalização da unidade).

Para o sistema de controle de presença e jornada, é necessário:

- Restringir a visualização de dados e aprovações estritamente pela fronteira do locatário (`organizacao_id`) e da lotação (`unidade_id`).
 - Permitir exceções individuais.
 - Filtrar os campos da resposta dinamicamente (ex: ocultar o CPF se o requisitante não possuir o perfil adequado).
-

3. Requisitos

3.1 Funcionais

1. Autorização por endpoint, método HTTP, atributos do recurso, usuário e unidade.
2. Suporte a políticas contextuais (horário oficial, geolocalização e biometria).
3. Resposta parcialmente filtrada baseada no perfil.
4. Auditoria completa e imutável de decisões críticas.

3.2 Não Funcionais

- Baixa latência (< 20ms por decisão de segurança).
 - Cache distribuído para avaliação de políticas locais.
 - Alta auditabilidade (Log estruturado).
 - **Conformidade LGPD:** As regras granulares devem usar a estratégia de *Fetch On-Demand* e evitar *Payloads* ricos no JWT.
-

4. Modelo de Autorização Proposto

4.1 Camadas de Controle

A arquitetura formaliza a remoção de acoplamento do Keycloak para autorização. Toda a decisão ocorre de forma tardia no backend.

Camada	Tipo	Componente Técnico Responsável
Autenticação e IAM	OIDC	Keycloak (Autentica a credencial técnica).
Autorização Organizacional	RBAC	Tabela <code>folha_ponto.vinculo_usuario</code> (PostgreSQL).
Autorização Granular	ABAC / PBAC	PDP Customizado consumindo a tabela <code>folha_ponto.excecao_permissao_usuario</code> .
Proteção de Atributos	Field-level filtering	Middleware interno / Serializadores JSON (Jackson Filters).

5. Modelo Híbrido

5.1 RBAC (Base Organizacional)

Roles consolidadas são armazenadas estritamente na tabela

`folha_ponto.vinculo_usuario`, definindo permissões amplas e amarradas a um `organizacao_id` e a um `usuario_id` (Identidade Canônica):

- OWNER
- ADMIN
- MANAGER
- PARTICIPANTE

5.2 ABAC (Atributos Dinâmicos)

O Motor de Políticas avalia os atributos em tempo real (dados internos do domínio):

- Identidade do Usuário (CPF e Confiança Biométrica).
- Relacionamento espacial (Geofence da unidade).
- Escopo Temporal (Horário do evento vs Horário do servidor).

5.3 PBAC (Policy-Based Access Control)

Políticas explícitas para exceções e casos extremos gravadas no banco de dados, utilizando a coluna `condicao_json` da tabela de exceções:

```
{
  "policy": "usuario_somente_sua_unidade",
  "effect": "permit",
  "condition": "usuario.unidade_id == recurso.unidade_id"
}
```

6. Arquitetura Técnica (Integração Segura)

A orquestração obedece à diretriz de Identidade Canônica em Três Camadas.

6.1 O Fluxo de Decisão

1. **Interceptação:** O usuário interage com o Frontend, e a API recebe a requisição contendo o *Lightweight Access Token* (com apenas a `claim sub`).
 2. **Conversão de Identidade:** O componente `customJwtAuthenticationConverter` intercepta o JWT, extrai o `sub`, e consulta a tabela `pessoa.identidade_acesso` para localizar o CPF da pessoa.
 3. **Resolução de Papéis:** O conversor busca em `folha_ponto.vinculo_usuario` as *Roles* locais daquele locatário.
 4. **Contexto de Segurança:** Um **`BusinessPrincipalToken`** (agora ciente do CPF real e dos papéis de negócio) é injetado no `Spring SecurityContextHolder`.
 5. **Avaliação Granular:** O *Policy Decision Point (PDP)* do Spring Boot é invocado (via anotações de método ou *interceptor*).
 6. **Consulta PBAC:** O PDP cruza as informações no cache ou busca regras exclusivas na tabela `folha_ponto.excecao_permissao_usuario`.
 7. **Desfecho:** O PDP retorna `PERMIT` ou `DENY` para a requisição.
-

7. Uso do Motor de Autorização Interno (PDP Customizado)

Ao invés de delegar avaliações complexas ao `Authorization Services` do Keycloak, a aplicação resolve suas permissões mapeando recursos contra o estado canônico do banco de dados local.

7.1 Recursos Modelados

Exemplo:

- **Recurso:** /presencas
 - **Escopos (Scopes):** view, edit, delete (bloqueado para eventos imutáveis), approve
-

8. Controle por Endpoint + Método

Mapeamento interno da aplicação:

Endpoint	Método	Scope Mapeado no PDP
/presencas	GET	view
/presencas	POST	create
/presencas/{id}	PUT	edit (Apenas metadados administrativos)
/presencas/{id}	DELETE	delete

O Middleware Spring Security traduz e envelopa a requisição HTTP para o respectivo escopo de avaliação.

9. Controle por Campo (Field-Level Security)

Após a decisão positiva de acesso (PERMIT), a aplicação pode ocultar dados sensíveis baseados na Role do `BusinessPrincipalToken`.

1. A política local aciona um filtro de resposta.
2. O Jackson (Serializador JSON) restringe o payload final de entrega.

Exemplo Prático: Uma consulta de listagem retornará os hashes biométricos criptografados ou o endereço residencial *somente* se o usuário que solicitou possuir a função atestada de AUDITOR ou OWNER. Para papéis inferiores, o backend emitirá o JSON suprimindo sumariamente essas colunas sensíveis.

10. Controle por Usuário Individual (Exceções e Overrides)

Para conceder privilégios temporários sem alterar a *Role* global do usuário (ex: um SERVIDOR que atua temporariamente como aprovador em um final de semana específico), o Motor de Políticas avalia fisicamente a tabela **folha_ponto.excecao_permissao_usuario**.

```
SELECT * FROM folha_ponto.excecao_permissao_usuario
WHERE usuario_id = ?
      AND organizacao_id = ?
      AND recurso = '/presencas'
      AND escopo = 'approve';
```

O uso desta tabela (que possui chave UUID e auditoria por locatário) garante a granularidade extrema e evita "gambiarras" através da criação de grupos descartáveis no Active Directory.

11. Contextualização por Geolocalização e IP

As condições armazenadas na coluna `condicao_json` podem ser utilizadas pelo PDP para invocar validações geoespaciais em tempo real com o PostGIS:

```
if (request.ip() in policy.allowedSubnets)
if (businessPrincipal.currentLocation() within unidade.poligono_geo())
```

Isso permite impor limites corporativos, como:

- Restringir aprovações de jornada a equipamentos que estejam conectados exclusivamente nas redes do órgão governamental.
-

12. Auditoria e Rastreabilidade

O paradigma de "Zero Trust" exige que **toda e qualquer** barreira de acesso negada pelo PDP gere uma evidência irrevogável na tabela `folha_ponto.log_auditoria`, registrando:

- `usuario_id` e `organizacao_id`
 - `acao` (endpoint e método)
 - `entidade_id` (se aplicável)
 - `decisao` (DENY)
 - `metadados` (JSON com os detalhes da política rejeitada)
 - `criado_em` (Timestamp oficial do servidor)
-

13. Estratégia de Performance

13.1 Cache Distribuído

- Como a conversão de Identidade (`identidade_acesso`) e as regras da `execcao_permissao_usuario` são lidas frequentemente pelo `customJwtAuthenticationConverter`, o uso de Cache de 2º Nível (ex: Redis ou Caffeine) é obrigatório.
 - **Chave de Cache Estrutural:** `hash(sub + org_id + resource + scope)`
 - **Vigência (TTL):** Curto (15 a 60 segundos), visando a garantia de que demissões e suspensões tenham a revogação efetivada quase em tempo real (Near Real-Time Revocation).
-

14. Modelo de Banco (Permissões Dinâmicas)

Conforme materializado nas DDLs da RFC-0005, o sustentáculo do PBAC reside no schema multi-tenant, isolado com a sua `organizacao_id`. O armazenamento de lógicas condicionais complexas no tipo `jsonb` do PostgreSQL viabiliza que as regras de controle sejam estendidas sem a necessidade de reestruturação do banco ou alteração do modelo relacional no Java.

15. Segurança (Lightweight Tokens e Fetch On-Demand)

- **Fetch On-Demand:** Jamais delegar ou ler permissões complexas, roles organizacionais de negócio ou regras de exceção a partir das *claims* de um JWT. O token será estritamente *Lightweight*, cego para as regras de negócio.
- **Proteção contra Adulteração:** Todo o contexto de segurança e PBAC se constrói dentro das memórias blindadas do Backend a partir da leitura confiável das tabelas da arquitetura relacional.
- **Criptografia OIDC:** As garantias primárias de acesso iniciam-se com o Spring Boot inspecionando criptograficamente a validade matemática (RS256) das chaves assimétricas emitidas pelo Keycloak antes que a esteira de PBAC comece a atuar.

16. Conclusão

A arquitetura proposta garante uma avaliação refinada por endpoint, método HTTP, nível de campo, usuário e contexto ambiental sem nenhuma dependência operacional de middlewares complexos de autorização alheios ao contexto do negócio.

A centralização da Política Dinâmica de Acesso (PBAC) na tabela de Exceções do PostgreSQL permite uma fachada de API escalável, auditável de forma granular e inegociavelmente isolada conforme os padrões da LGPD.